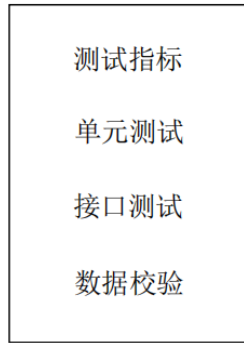


Web前后端通信与测试技术



前后端交互



测试



实验: 编写 Web App 测试

1.HTTP

1.1 HTTP请求响应模型

- 建立连接：客户端和服务器之间建立连接。
- 发送请求：客户端向服务器发送请求，请求包含要访问的资源的URL，请求方法、请求头以及可选的请求体。
- 处理请求：服务器收到请求后，根据请求中的信息找到相应的资源执行处理操作。
- 发送响应：服务器将处理后的结果封装在响应中，并发送回客户端。响应包含状态码、响应头（例如：content-type,content-length）以及可选的响应体。
- 关闭连接

1.2 HTTP方法

- GET：请求从服务器获取指定资源，用于访问页面
- POST：请求服务器接收并处理请求体中的数据，通常用于表单提交
- PUT：请求服务器存储一个资源，并用请求体中的内容替换目标资源的内容
- DELETE：请求服务器删除指定的资源
- HEAD:与GET类似，但不获取资源的内容，只获取响应头信息
- OPTIONS：返回服务器支持的HTTP方法，用于检查服务器支持哪些请求方法，通常用于CORS的预检请求
- TRACE：回显服务器收到的请求，用于诊断
- CONNECT：建立一个到服务器的隧道，通常用于HTTPS连接。客户端可以通过该隧道发送加密的数据

1.3 HTTP状态码

- 1xx:信息，服务器收到请求，需要请求者继续执行操作
- 2xx：成功，操作被成功接收并处理
- 3xx：重定向，需要进一步的操作
- 4xx：客户端错误，请求包含语法错误或无法完成请求
- 5xx:服务器错误，服务器在处理请求的过程中发生了错误

1.4 HTTP响应头信息

- Date
- Server:服务器
- Content-Type:内容类型
- Content-Length:内容长度
- Content-Coding: 内容编码
- Content-Language:内容语言
- Content-Location:响应体的URI
- 等等

1.5 HTTP RESTful

REST (Representational State Transfer) :表述性状态转移。REST的基本思想是通过公认的、与语言无关的、可靠标准化的客户端/服务器交互传输资源（例如文档）。REST指的是一组架构约束条件和原则，如果一个架构满足REST，则成为RESTful架构。

REST中的表述，其实指的是资源。资源的唯一标识就是URI。

- ? 可用来过滤资源。
- , 或; 表示同级资源的关系。

1.6 HTTP请求参数

HTTP请求参数允许你将信息从客户端传输到服务器端的Web应用程序。它们通常用于提供条件查询、过滤结果或指定应返回哪些数据。

Router Param

如在<http://example.com/api/users/123>中，123是一个router param。

Query String

位于?后面，各参数用&分隔，如在<http://example.com/api/products?category=books&price=low>中，`category` 和 `price` 是查询字符串参数，用于筛选类别为图书且价格低廉的产品。

Body Param

不在URL中显示，而是在HTTP请求体部分发送，适用于发送大量数据。

2.axios

axios是一个基于promise可以用于浏览器和node.js的网络请求库。它是isomorphic的（即同一套代码可以运行在浏览器和Node.js中）。在服务端使用原生Node.js http模块，在客户端使用XMLHttpRequests。

[axios中文文档](#)

3.CORS

在前端代码调用后端服务中经常会碰到下面的错误，这就是最为常见的跨域CORS错误。

```
1 | Access to fetch at 'http://127.0.0.1:7002/' from origin  
   | 'http://127.0.0.1:7001' has been blocked by CORS policy: No 'Access-Control-  
   | Allow-Origin' header is present on the requested resource. If an opaque  
   | response serves your needs, set the request's mode to 'no-cors' to fetch the  
   | resource with CORS disabled.
```

出于安全性，浏览器限制脚本内发起的跨源HTTP请求。例如，XMLHttpRequest和Fetch API遵循同源策略。这意味着使用这些API的web应用程序只能从加载应用程序的同一个域请求HTTP资源，除非响应报文包含正确CORS响应头。

CORS机制允许Web应用服务器进行跨源访问控制，从而使跨源数据传输得以安全进行。现代浏览器支持在API容器中使用CORS。

[跨域](#)

4.WebSocket

WebSocket是HTML5开始提供的一种在单个TCP连接上进行全双工通讯的协议。WS使得客户端和服务端之间的数据交换变得更加简单，允许服务端主动向客户端推送数据。在WS API中，浏览器和服务端只需要完成一次握手，两者之间就可以直接创建持久性的连接，并进行双向数据传输。

[WebSocket教程](#)

[WebSocket详解](#)

5.测试

5.1 前端测试Vitest

Vitest是由Vite驱动的下一代测试框架。

[Vitest中文文档](#)

5.2 后端测试Jest

Jest是一个JS测试框架。

[Jest中文文档](#)

5.3 断点调试